AicoreLink

Plan: AicoreLink SaaS Redesign — Sitios Web con IA para Negocios Locales

Fecha: 2026-10-11 **Spec:** `docs/superpowers/specs/2026-10-11-aicorelink-saas-redesign-design.md` **Objetivo:** Transformar el scaffold actual en una plataforma SaaS B2B completa para vender sitios web con IA a negocios locales (3 planes, suscripción mensual, provisión automática, portal cliente, IA + WhatsApp + Voz). **Estado:** Guardado para implementación futura. NO es el plan activo (el activo es solo la Landing).

---

1. Arquitectura y Decisiones Técnicas Clave

Área	Decisión	Rationale
Monorepo	`apps/dashboard` único (marketing + portal + API)	Simplicidad deploy, shared code, single DB
Multi-tenancy	PostgreSQL compartida + `tenant_id` + RLS	Aislamiento datos, escalabilidad, un solo schema
Auth	NextAuth v4 Credentials + JWT + middleware	Integrado Next.js, roles `admin` / `cliente` / `dev_aicore`
Provisión tenant	Webhook Stripe → `POST /api/tenants/provision` → crea tenant + subdominio + n8n workflow + envía credenciales	Automatización completa, zero-touch onboarding
Subdominios	`cliente.aicorelink.aicorebots.com` (wildcard DNS + Traefik)	SSL automático, aislamiento, branding
IA	Ollama (Mistral local) primary → OpenRouter (GLM) fallback	Costo controlado, privacidad, latencia baja
WhatsApp	Meta Cloud API (un número compartido por tenant via webhook `tenant_id`)	Costo bajo, gestión centralizada
Voz	Whisper (STT) → IA → ElevenLabs (TTS) → nota de voz WhatsApp	Pipeline modular, cada pieza intercambiable
Pagos	Stripe Subscriptions + Setup Fee + Usage-based (overages WhatsApp/IA)	Modelo SaaS estándar, métricas built-in
Deploy	GitHub Actions → ghcr.io → Dokploy (VPS 51.222.207.250)	Pipeline ya validado, rollback fácil

---

2. Restricciones Globales (del Spec)

- **Nunca** commitear secrets (`.env`, keys) → Dokploy Environment
- **Siempre** `withTenant()` para queries portal cliente (RLS)
- **Siempre** validar entrada con Zod en API routes
- **Siempre** loguear acciones sensibles en `audit_log`
- **Ask First** antes de: cambios `prisma/schema.prisma`, nuevas deps npm, modificar `next.config.mjs` / `Dockerfile`, cambiar RLS, cambiar proveedores IA
- **Never** desactivar tests que fallan sin fixear root cause
- **Never** hardcodear URLs producción en código
- **Never** usar `BYPASSRLS` en rol aplicación (solo admin)

---

3. Focos de Review (5 Failure Modes)

1. **Multi-tenancy leaks** — Queries sin `tenant_id`, RLS policies faltantes, cross-tenant data access
2. **Provisión race conditions** — Stripe webhook duplicado, tenant creado 2x, subdominio collision
3. **IA cost runaway** — Sin límites usage por plan, fallback OpenRouter sin budget, tokens no trackeados
4. **WhatsApp webhook security** — Signature verification faltante, replay attacks, tenant confusion
5. **Deploy drift** — Dockerfile vs local dev divergence, env vars missing en Dokploy, migration order

---

FASE 1: Marketing Site Público (Prioridad Alta — 2-3 semanas)

1.1 Landing Page — `aicorelink.aicorebots.com`

Task	Archivos	Criterio de Aceptación	Test
1.1.1 Layout marketing `(marketing)/layout.tsx`	`app/(marketing)/layout.tsx`	Header minimal (logo + CTA), footer con links ecosistema, `Metadata` SEO base	E2E: carga sin error, meta tags presentes
1.1.2 Hero Section	`components/marketing/HeroSection.tsx`	Headline + subhead + CTA primario/secundario + trust signal + demo modal trigger	Unit: render correcto; E2E: CTA navega a `/planes`
1.1.3 Pain Cards (5 dolores)	`components/marketing/PainCard.tsx`, `PainGrid.tsx`	5 tarjetas con icono animado, copy persuasivo, hover micro-interacción	Visual regression; a11y: focus visible
1.1.4 Solution Reveal (3 columnas)	`components/marketing/SolutionReveal.tsx`	Atrae → Convierte → Retiene con iconos + copy	Unit: structure correcta
1.1.5 Pricing Table interactiva	`components/marketing/PricingTable.tsx`, `PricingCard.tsx`	Toggle mensual/anual (20% desc), 3 planes, plan Growth destacado (badge teal + glow), CTA por plan	Unit: toggle funciona; E2E: CTA → `/register?plan=X`
1.1.6 Vertical Tabs + Demo Interactiva	`components/marketing/VerticalTabs.tsx`, `InteractiveDemo.tsx`	Tabs por vertical (Clínicas, Gimnasios, Restaurantes, Inmobiliarias, Servicios), cada tab: screenshot + métricas + botón "Ver demo" abre modal con chat real funcionando	E2E: switch tabs sin reload, demo modal abre y chat responde
1.1.7 FAQ (Objection Handling)	`components/marketing/FAQ.tsx`	5 preguntas acordeón, copy del spec	Unit: acordeón abre/cierra
1.1.8 CTA Final + Trust Bar	`components/marketing/CTAFinal.tsx`, `TrustBar.tsx`	"¿Listo para dejar de perder clientes?" + botón + link Calendly + logos clientes + métricas agregadas	E2E: CTA navega, links externos abren nueva pestaña
1.1.9 Page principal `/` ensambla todo	`app/(marketing)/page.tsx`	Composición ordenada: Hero → Pain → Solution → Pricing → VerticalTabs → FAQ → CTA	E2E: scroll fluido, todas las secciones renderizan

1.2 Página Planes Detallada `/planes`

Task	Archivos	Criterio de Aceptación	Test
1.2.1 Layout + Metadata	`app/(marketing)/planes/page.tsx`	SEO: title "Planes y Precios - AicoreLink", description, OG tags	Unit: metadata export
1.2.2 Tabla comparativa extendida	`components/marketing/PlansComparison.tsx`	Filas: páginas, formulario, chat IA, calendario, reseñas, voz, setup fee, precio mensual/anual, CTA	Visual regression desktop/mobile
1.2.3 FAQ específico planes	`components/marketing/PlansFAQ.tsx`	6-8 preguntas: límites, overages, dominio propio, migración, soporte, cancelación	Unit: render

1.3 Registro + Checkout Stripe

Task	Archivos	Criterio de Aceptación	Test
1.3.1 Página registro `/register`	`app/(marketing)/register/page.tsx`	Form: email, password, nombre negocio, plan (query param), TOS checkbox; Zod validation	Unit: validation; E2E: submit crea usuario + redirect checkout
1.3.2 API `/api/auth/register`	`app/api/auth/register/route.ts`	Crea user en BD (role `cliente`), tenant en `pending`, inicia Stripe Checkout Session con `metadata: { tenantId, plan }`	Integration: user + tenant creados, session URL válida
1.3.3 Stripe Checkout Success `/register/success`	`app/(marketing)/register/success/page.tsx`	Muestra "Provisionando tu cuenta..." + polling a `/api/tenants/status?tenantId=X`	E2E: polling completa → redirect portal
1.3.4 Webhook Stripe `checkout.session.completed`	`app/api/webhooks/stripe/route.ts`	Verifica signature, llama `provisionTenant(tenantId, plan)`, maneja idempotencia (event id)	Integration: tenant provisionado, subdominio DNS, n8n workflow, email enviado
1.3.5 Provisión automática tenant	`lib/tenants/provision.ts`	Transacción: actualiza tenant `status=active`, crea subdominio (wildcard DNS ya configurado), genera `META_VERIFY_TOKEN`, crea workflow n8n vía API, envía email credenciales	Integration: todo creado, email llega, subdominio responde 200
1.3.6 Email bienvenida + credenciales	`lib/email/welcome.ts` (Resend/SendGrid)	Template HTML: bienvenida, link portal (`cliente.aicorelink.aicorebots.com`), credenciales temporales, next steps	Unit: template render; Integration: email enviado

1.4 Componentes UI Base (Marketing)

Task	Archivos	Criterio de Aceptación
1.4.1 Primitivos extendidos	`components/ui/` (Button, Card, Input, Modal, Badge, Tabs, Accordion, Toggle, Tooltip)	Consistencia con ecosistema: `bg-brand`, `text-navy`, `rounded-lg`, `focus-visible:ring-brand`, Framer Motion `whileHover/whileTap`
1.4.2 Animaciones globales	`components/ui/animations.ts`	`fadeInUp`, `staggerContainer`, `staggerItem`, `hoverLift`, `tapScale` — reutilizables
1.4.3 Iconos (lucide-react)	`components/ui/icons.tsx`	Wrapper consistente `size={24}`, `strokeWidth={2}`, `className` passthrough

1.5 Configuración Técnica Marketing

Task	Archivos	Criterio de Aceptación
1.5.1 `next.config.mjs` marketing	`next.config.mjs`	`output: 'standalone'`, `images.remotePatterns` para screenshots verticales, `headers()` para CSP, `rewrites()` para subdominios futuros
1.5.2 Middleware marketing	`middleware.ts`	No auth en `(marketing)/*`, redirect `/` → `/` (no-op), headers seguridad (HSTS, X-Frame-Options)
1.5.3 Sitemap + robots	`app/(marketing)/sitemap.ts`, `robots.ts`	Generación dinámica estática, incluye `/planes`, `/register`, verticales
1.5.4 Analytics + Web Vitals	`lib/analytics.ts`	Vercel Analytics / Plausible / GA4 opcional, `reportWebVitals` en `_document.tsx`

FASE 2: Portal Cliente (Prioridad Alta — 3-4 semanas)

2.1 Layout + Auth Portal

Task	Archivos	Criterio de Aceptación
2.1.1 Layout portal `(portal)/layout.tsx`	`app/(portal)/layout.tsx`	Sidebar responsive (60px collapsed / 240px expanded), header con avatar + tenant selector (si multi-tenant user), navigation: Dashboard, WhatsApp, Sitio, Leads, Facturación, Configuración
2.1.2 Middleware portal auth	`middleware.ts` (extendido)	Protege `(portal)/*` → redirect `/login` si no autenticado, inyecta `tenantId` en headers para RLS
2.1.3 Login page `/login`	`app/login/page.tsx`	Form email/password, "Recordarme", link "Olvidé contraseña", redirect post-login a `/dashboard`
2.1.4 Password reset flow	`app/(auth)/reset-password/`, `api/auth/reset/`	Token JWT 1h, form nueva password, Zod validation

2.2 Dashboard Métricas

Task	Archivos	Criterio de Aceptación
2.2.1 API métricas `/api/tenants/[id]/metrics`	`app/api/tenants/[id]/metrics/route.ts`	`GET` con `withTenant()`: leads totales/mes, conversaciones activas, uso WhatsApp (msg/mes), uso IA (tokens/mes), costo estimado, health score
2.2.2 MetricCard components	`components/portal/MetricCard.tsx`, `UsageMeter.tsx`	Valor grande + trend (↑↓%) + sparkline (recharts) + barra circular progreso (uso/límite plan)
2.2.3 Dashboard page	`app/(portal)/dashboard/page.tsx`	Grid 4 MetricCards + gráfico leads últimos 30 días + alerts (sin leads 14d, uso IA < 20%, base conocimiento vacía)

2.3 Configuración WhatsApp + IA

Task	Archivos	Criterio de Aceptación
2.3.1 Page WhatsApp	`app/(portal)/whatsapp/page.tsx`	Tabs laterales: General / IA / Escalado / Voz; preview derecho con chat demo funcional
2.3.2 Tab General	`components/portal/whatsapp/GeneralTab.tsx`	Nombre negocio, horario atención, mensaje bienvenida, mensaje fuera horario, número WhatsApp (display only), webhook URL (copy button), `META_VERIFY_TOKEN` (copy)
2.3.3 Tab IA (Base Conocimiento + Intents)	`components/portal/whatsapp/AITab.tsx`	Editor base conocimiento (markdown + bloques Q&A), selector modelo (Ollama/OpenRouter), temperature, max tokens, intents personalizados (nombre + palabras clave + respuesta)
2.3.4 Tab Escalado	`components/portal/whatsapp/EscalationTab.tsx`	Palabras clave escalado (array), email/Telegram notificación, horario escalado, mensaje "te transfiero con humano"
2.3.5 Tab Voz	`components/portal/whatsapp/VoiceTab.tsx`	Toggle habilitar voz, selector voz ElevenLabs (preview audio), idioma, mensaje bienvenida voz, fallback STT error
2.3.6 API canales `/api/channels/[id]`	`app/api/channels/[id]/route.ts`	`GET` / `PATCH` con `withTenant()`, valida Zod, actualiza config, dispara regeneración workflow n8n
2.3.7 Chat Demo Preview	`components/portal/whatsapp/ChatPreview.tsx`	Iframe o modal con n8n webhook test endpoint, simula conversación real

2.4 CRM Simple (Leads + Conversaciones)

Task	Archivos	Criterio de Aceptación
2.4.1 API leads `/api/leads`	`app/api/leads/route.ts`	`GET` (paginado, filtros: fecha, estado, source), `POST` (webhook n8n crea lead), `GET /:id` (detalle + conversación completa)
2.4.2 LeadTable component	`components/portal/LeadTable.tsx`	TanStack Table: sortable, filtrable, row-expand muestra conversación, exportar CSV, badge estado (nuevo, en conversación, calificado, convertido, perdido)
2.4.3 Leads page	`app/(portal)/leads/page.tsx`	Header con stats + LeadTable + empty state CTA "Ver demo"
2.4.4 Conversación detail modal	`components/portal/ConversationModal.tsx`	Timeline mensajes (usuario/IA/humano), timestamps, metadata (intent detectado, escalado, tokens usados)

2.5 Facturación + Stripe Portal

Task	Archivos	Criterio de Aceptación
2.5.1 API facturación `/api/billing`	`app/api/billing/route.ts`	`GET`: suscripción actual, historial invoices (Stripe), próximo cobro, usage actual vs límite
2.5.2 Stripe Customer Portal	`app/api/billing/portal/route.ts`	`POST` crea `billing_portal.session` → redirect URL
2.5.3 Facturación page	`app/(portal)/facturacion/page.tsx`	Plan actual + badge, próxima factura, tabla historial (fecha, monto, estado, link PDF), botón "Gestionar suscripción" → Stripe Portal, medidor uso WhatsApp/IA

2.6 Configuración Equipo + Perfil

Task	Archivos	Criterio de Aceptación
2.6.1 API team `/api/tenants/[id]/members`	`app/api/tenants/[id]/members/route.ts`	`GET` lista miembros, `POST` invita (email + role), `DELETE` remueve, `PATCH` cambia role
2.6.2 Team page	`app/(portal)/configuracion/equipo/page.tsx`	Tabla miembros, botón "Invitar", modal invite, roles: `admin` (dueño), `agente` (responde chats), `viewer` (solo lectura)
2.6.3 Perfil page	`app/(portal)/configuracion/perfil/page.tsx`	Nombre, email, avatar (upload → S3/Cloudinary), cambio password, notificaciones email/Telegram

FASE 3: IA + WhatsApp + Voz (Prioridad Media — 4-5 semanas)

3.1 Clasificador Intención + RAG

Task	Archivos	Criterio de Aceptación
3.1.1 Ollama client + fallback	`lib/ai/ollama-client.ts`, `lib/ai/openrouter-client.ts`	`classifyIntent(message, baseConocimiento)` → `{ intent, confidence, response }`, fallback automático OpenRouter si Ollama error/timeout > 3s
3.1.2 RAG simple (embeddings + similarity)	`lib/ai/rag.ts`	Chunk base conocimiento (markdown → paragraphs), embeddings Ollama `nomic-embed-text`, vector search (pgvector o cosine similarity en memoria), top-k 3 contextos
3.1.3 Prompt engineering system	`lib/ai/prompts.ts`	System prompt por tenant: rol, tono, base conocimiento, reglas escalado, info negocio; template injection seguro
3.1.4 n8n workflow clasificación	`n8n-workflows/whatsapp-text.json`	Webhook → Clasifica → Si confidence > 0.8 responde → Si < 0.8 escala → Log conversación → Guarda lead si nuevo
3.1.5 API callback n8n `/api/webhooks/n8n`	`app/api/webhooks/n8n/route.ts`	Recibe clasificación, actualiza conversación, dispara notificación si escalado, responde WhatsApp vía Meta API

3.2 WhatsApp Voz (STT → IA → TTS)

Task	Archivos	Criterio de Aceptación
3.2.1 Whisper STT client	`lib/voice/stt.ts`	`transcribe(audioBuffer)` → `{ text, language, duration }`, timeout 10s, fallback mensaje "no pude entender"
3.2.2 ElevenLabs TTS client	`lib/voice/tts.ts`	`synthesize(text, voiceId)` → `audioBuffer` (mp3), cache por hash(text+voiceId) 24h
3.2.3 Pipeline voz n8n	`n8n-workflows/whatsapp-voice.json`	Webhook audio → Whisper → Clasificador IA → TTS → Responde nota de voz WhatsApp
3.2.4 API Whisper callback	`app/api/webhooks/whatsapp/route.ts` (extendido)	Detecta `audio` message type → descarga media → pipeline voz → respuesta

3.3 Reseñas Automáticas

Task	Archivos	Criterio de Aceptación
3.3.1 Trigger post-conversación	`n8n-workflows/review-automation.json`	Espera 30min post-último mensaje → si conversación > 3 mensajes y no escalada → envía WhatsApp con link Google Review personalizado (UTM)
3.3.2 Link Google Review generator	`lib/reviews/google-review.ts`	`generateReviewUrl(tenantId, leadId)` → URL con `place_id` + `review=` prefill
3.3.3 Tracking reseñas	`lib/reviews/track.ts`	Webhook Google (si disponible) o polling diario → actualiza `lead.reviewed = true`, métrica `reviews_generated`

3.4 Límites + Usage Tracking + Billing Overages

Task	Archivos	Criterio de Aceptación
3.4.1 Entitlements por plan	`lib/entitlements.ts`	`hasFeature(tenantId, 'voice')`, `getLimit(tenantId, 'whatsapp_msgs_month')`, `getLimit(tenantId, 'ai_tokens_month')`
3.4.2 Usage tracker middleware	`lib/usage/tracker.ts`	`trackUsage(tenantId, 'whatsapp_outbound', 1)`, `trackUsage(tenantId, 'ai_tokens', tokens)` → upsert `usage_log` diario
3.4.3 Overages billing job	`lib/billing/overages.ts`	Cron diario: agrega usage mensual, si > límite plan → crea `invoice_item` Stripe (precio configurado por plan)
3.4.4 Alertas uso (portal + email)	`components/portal/UsageAlerts.tsx`	Banner en dashboard si uso > 80% límite, email a admin tenant si > 95%

FASE 4: Site Builder (Opcional / MVP+ — 4-6 semanas)

> **Nota:** Fase 4 es opcional para MVP. Decidir tras validar Fase 1-3 con clientes reales.

Task	Archivos	Criterio de Aceptación
4.1 Editor visual drag-drop	`app/(portal)/sitio/editor/page.tsx`	Canvas con secciones predefinidas (Hero, Servicios, Testimonios, Contacto, Mapa, FAQ), sidebar props, preview responsive (desktop/tablet/mobile)
4.2 Secciones predefinidas	`components/site-builder/sections/`	10+ secciones con props tipadas (Zod), validación required fields, variants (estilo A/B/C)
4.3 Motor renderizado sitio	`lib/sites/renderer.tsx`	Server Component que recibe JSON sitio → renderiza secciones en orden, inyecta JSON-LD, meta tags, sitemap
4.4 Publicación subdominio	`lib/sites/publish.ts`	`publish(tenantId, siteJson)` → escribe archivos estáticos en storage (S3/VPS) → configura Traefik routing → invalida CDN cache
4.5 SEO automático	`lib/sites/seo.ts`	Genera: `title`, `description`, `og:*`, `twitter:*`, `json-ld` (LocalBusiness, WebSite), `sitemap.xml`, `robots.txt`
4.6 API sitios `/api/sites`	`app/api/sites/route.ts`	CRUD sitio JSON, `POST /publish` dispara publicación, `GET /preview` render temporal

FASE 5: Growth & Ops (Continua — Post-Launch)

Task	Archivos	Criterio de Aceptación
5.1 Admin Dashboard multi-tenant	`app/(admin)/admin/page.tsx`	MRR, ARR, churn rate, LTV, CAC, tenants activos, trials, usage agregado, revenue por plan
5.2 Onboarding automatizado	`lib/onboarding/sequence.ts`	Email sequence (D1 bienvenida, D3 setup WhatsApp, D7 primer lead, D14 revisión), checklist en portal, webhook n8n "primer lead" → celebra
5.3 Programa referidos	`lib/referrals/`	Código único por tenant, 1 mes gratis por referido que pague 3 meses, tracking `referral_code` en Stripe metadata
5.4 A/B Testing landing	`lib/experiments/`	Middleware asigna variante (A/B) por cookie, métricas: CTA click, signup, checkout complete; dashboard resultados
5.5 Health Monitoring interno	`lib/monitoring/`	Uptime checks (marketing, portal, API, n8n, DB), alertas Slack/Telegram si degradado, SLO dashboards

4. Orden de Dependencias (Critical Path)

```
FASE 1 (Marketing)
├── 1.1 Landing (1.1.1–1.1.9) ──┐
├── 1.2 /planes (1.2.1–1.2.3)   ├── Paralelizables tras 1.1.1
├── 1.4 UI Primitivos (1.4.1–1.4.3) ──┤ (base para todo)
├── 1.5 Config Técnica (1.5.1–1.5.4) ──┘
└── 1.3 Checkout Stripe (1.3.1–1.3.6) ← Requiere 1.1.5 (CTAs), 1.4.1 (Button/Modal)

FASE 2 (Portal)
├── 2.1 Layout + Auth (2.1.1–2.1.4) ──┐
├── 2.2 Dashboard (2.2.1–2.2.3)       ├── Paralelizables tras 2.1.2
├── 2.3 WhatsApp Config (2.3.1–2.3.7) ├── Requieren 1.3.5 (tenant provisionado)
├── 2.4 CRM Leads (2.4.1–2.4.4)       │
├── 2.5 Facturación (2.5.1–2.5.3)     │
└── 2.6 Equipo/Perfil (2.6.1–2.6.3)   └── Requieren 2.1.1 (layout)

FASE 3 (IA + Voz + Reseñas)
├── 3.1 Clasificador + RAG (3.1.1–3.1.5) ──┐
├── 3.2 Voz (3.2.1–3.2.4)                  ├── Paralelizables
├── 3.3 Reseñas (3.3.1–3.3.3)              │   Requieren 3.1.4 (n8n workflow base)
└── 3.4 Usage + Overages (3.4.1–3.4.4)     └── Requieren 2.5.1 (billing base)

FASE 4 (Site Builder) ← Requiere FASE 1–3 estables + clientes reales validando

FASE 5 (Growth) ← Continua, paralela a todo
```

5. Estimación de Esfuerzo (Días-Hombre)

Fase	Tasks	Días Estimados	Paralelismo	Calendar Days (1 dev)
Fase 1	22 tasks	~25 días	Alto (UI + API separadas)	3-4 semanas
Fase 2	20 tasks	~30 días	Medio (auth base compartida)	4-5 semanas
Fase 3	16 tasks	~28 días	Medio (n8n + IA + voz independientes)	4-5 semanas
Fase 4	6 tasks	~35 días	Bajo (editor complejo)	5-6 semanas (opcional)
Fase 5	5 tasks	~15 días	Continuo	Ongoing

Total MVP (Fases 1-3): ~83 días-hombre → ~12-14 semanas (1 dev full-time)

6. Definition of Done por Fase

Fase 1 Done:

- [ ] Landing responde 200 en `aicorelink.aicorebots.com` con todas las secciones
- [ ] `/planes` muestra tabla comparativa funcional toggle mensual/anual
- [ ] Registro → Checkout Stripe → Provisión automática tenant + subdominio + n8n + email en < 3 min
- [ ] Subdominio `demo.aicorelink.aicorebots.com` carga portal login
- [ ] Lint + Typecheck + Tests (unit + integration + E2E critical path) pasan en CI

Fase 2 Done:

- [ ] Login portal funciona con credentials + JWT
- [ ] Dashboard muestra métricas reales (leads, conversaciones, usage, costo estimado)
- [ ] Config WhatsApp: guarda config, regenera n8n workflow, preview chat funciona
- [ ] CRM: lista leads, detalle conversación, export CSV
- [ ] Facturación: historial Stripe, Stripe Portal link, usage meters
- [ ] Equipo: invitar/remover/cambiar roles funciona

Fase 3 Done:

- [ ] Clasificador intención responde > 80% accuracy en test set vertical
- [ ] RAG: responde usando base conocimiento del tenant (no alucina)
- [ ] Escalado: palabras clave disparan notificación email/Telegram
- [ ] Voz: nota de voz entrante → respuesta nota de voz saliente < 5s p95
- [ ] Reseñas: link Google Review enviado post-conversación, tracking funciona
- [ ] Overages: billing job crea invoice items Stripe correctos

---

7. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Stripe webhook duplicado crea tenant 2x	Media	Alto	Idempotency key en webhook (Stripe event ID), `tenant.status` check antes de provisionar
Subdominio collision (cliente existente)	Baja	Alto	Validación unique en BD + check DNS antes de crear, slug generado: `slugify(nombre) + '-' + random(4)`
Costos IA/OpenRouter descontrolados	Media	Alto	Hard limits por plan en `entitlements.ts`, alertas 80%/95%, fallback Ollama only si overage
n8n workflow roto tras deploy	Media	Medio	Versionar workflows en `n8n-workflows/`, test automatizado post-deploy (health check webhook)
Meta WhatsApp API rate limits	Media	Medio	Queue en n8n con retry exponencial, batch outbound, monitor `x-rate-limit-remaining`
RLS policy gap expone data cross-tenant	Baja	Crítico	Test automatizado: `SELECT * FROM leads WHERE tenant_id ≠ current_tenant` debe retornar 0 rows

---

8. Próximos Pasos Inmediatos

1. **Aprobar este plan** → Confirmar alcance Fase 1 (¿todo? ¿subset?)
2. **Crear tasks atómicas** (skill `planning-and-task-breakdown`) para Fase 1 con:
   - Archivos exactos a crear/modificar
   - Interfaces TypeScript / Zod schemas
   - Test cases específicos (input → expected output)
   - Commit messages convencionales sugeridos
3. **Setup proyecto**: Verificar Dockerfile, Prisma schema, RLS, seed actualizados para nueva estructura (planes, suscripciones, usage_log, audit_log)
4. **Iniciar Task 1.1.1** → Layout marketing